

Machine Learning Based Network Intrusion Detection and Classification System

**KONERU LAKSHMAIAH EDUCATION FOUNDATION
(KLH UNIVERSITY, HYDERABAD)**

Department of Computer Science and Engineering

MACHINE LEARNING
BACHELOR OF TECHNOLOGY
A.Y.: 2026-27

A PROJECT REPORT

Submitted by

S. Divya — 2520039622
T Himabindu — 2520030484
R Rohith Varma — 2520030070

Under the Guidance of

Dr. N. Nirmalajyothi
Associate Professor, CSE

ABSTRACT

Network Intrusion Detection Systems (NIDS) are a core defensive control against the growing volume and sophistication of attacks on modern networks. Signature-based and purely rule-driven detection approaches struggle to keep pace with evolving traffic patterns and cannot reliably flag previously unseen attack behaviour, which motivates the use of data-driven, machine-learning-based classification. This project develops and evaluates a supervised machine-learning pipeline for multiclass network intrusion detection using the CIC-IDS2017 benchmark dataset, comprising 1,233,163 raw network-flow records described by 79 attributes and labelled into four traffic classes: BENIGN, Bot, DDoS, and Port-Scan. The pipeline performs systematic data-quality inspection, duplicate and missing-value handling, stratified train/test partitioning, training-derived IQR-based outlier capping, correlation-based redundancy filtering, feature standardization, and a three-stage feature-selection process (variance thresholding, a correlation recheck, and Random Forest importance ranking) that reduces the working feature space from 78 raw attributes to 12 final predictors.

Four classification algorithms — Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting — were trained on the reduced feature set and compared using accuracy, weighted precision/recall/F1, and Macro F1, the last of which was emphasised throughout because the dataset is heavily imbalanced (the BENIGN class alone accounts for 76.57% of records, while Bot traffic represents only 0.16%). A Random Forest baseline was additionally tuned via Randomized Search-CV; the tuned configuration achieved a higher cross-validated Macro F1 (0.9505) than the untuned baseline (0.9465) but did not improve held-out test performance, a result reported transparently rather than concealed. Across the four compared algorithms, Gradient Boosting attained the highest test accuracy (99.928%) while the Decision Tree attained the highest Macro F1 (0.9475) at a small fraction of Gradient Boosting's training cost, and was therefore adopted as the preferred model for detailed final evaluation. The final Decision Tree reached 99.916% accuracy and a Macro F1 of approximately 0.947 on 223,737 held-out test samples, with near-perfect per-class performance on BENIGN, DDoS, and Port-Scan traffic and a markedly weaker F1 (approximately 0.79) on the rare Bot class, which alone accounted for a disproportionate share of the remaining classification errors. These findings demonstrate that a carefully engineered, leakage-aware classical machine-learning pipeline can achieve strong aggregate intrusion-detection performance on CIC-IDS2017 while also making explicit where such a system remains weakest — namely on severely under-represented attack classes — a distinction of practical significance for any deployment that must prioritise the detection of rare, high-impact threats over headline accuracy.

KEYWORDS

Network Intrusion Detection; Machine Learning; CIC-IDS2017; Multiclass Classification; Feature Selection; Class Imbalance; Random Forest; Decision Tree; Cybersecurity.

1. INTRODUCTION

1.1 Purpose of the Project

The continued expansion of networked infrastructure has been accompanied by a parallel growth in the volume, diversity, and sophistication of cyber-attacks. Distributed Denial-of-Service (DDoS) campaigns, reconnaissance activity such as port scanning, and malware-driven botnet traffic each pose distinct risks to the availability, confidentiality, and integrity of networked systems. Network Intrusion Detection Systems (NIDS) are deployed at network boundaries or within internal segments to continuously monitor traffic and flag activity that deviates from expected, benign behaviour. Traditional NIDS approaches rely heavily on signature matching against known attack patterns; while effective against previously catalogued threats, such systems are structurally unable to recognise novel or subtly modified attacks, and they require constant manual maintenance of signature databases as the threat landscape evolves.

Machine learning offers an alternative, data-driven paradigm in which a classification model learns to distinguish benign from malicious traffic directly from labelled examples of network flow statistics, without requiring hand-crafted signatures for every attack variant. This project is motivated by the practical need to evaluate how far a classical, interpretable machine-learning pipeline — built from openly available tools such as scikit-learn — can go in accurately classifying network traffic into multiple categories using a realistic, large-scale benchmark dataset. The intended stakeholders for such a system include network security operations teams, who require both high aggregate accuracy and reliable detection of rare but high-impact attack classes, and researchers who use CIC-IDS2017 as a standard point of comparison for new detection techniques. Because real network traffic is rarely composed of a single attack type, multiclass classification — rather than simple benign/attack binary classification — is emphasised throughout this project, since it more closely reflects the operational need to distinguish among different attack categories and prioritise response accordingly.

1.2 Problem Statement

The project addresses the following machine-learning problem: given a set of numerical statistics describing a single network flow (e.g., packet counts, byte counts, flow duration, inter-arrival timing, and TCP flag counts), predict which of four categories the flow belongs to:

- BENIGN — normal, non-malicious network traffic
- Bot — traffic generated by malware-infected hosts communicating with a command-and-control infrastructure or performing automated malicious actions
- DDoS — Distributed Denial-of-Service attack traffic intended to overwhelm a target
- Port-Scan — reconnaissance traffic that probes a target's open ports and services

The expected output of the trained model, for any given flow record, is a single predicted class label among the four listed above. Because the four classes occur with very different frequencies in real traffic (as confirmed by the dataset's own class distribution), the problem is treated explicitly as an imbalanced multiclass classification task rather than a balanced one, which has direct consequences for how models are trained, tuned, and evaluated in this project.

1.3 Existing Methods and Disadvantages

Conventional intrusion detection has historically relied on two broad approaches: signature-based detection, which matches observed traffic against a database of known attack patterns, and simple statistical or threshold-based anomaly

detection. Signature-based systems are precise against known threats but are, by construction, unable to detect zero-day or previously unseen attack variants, and they demand continuous manual signature curation. Statistical threshold methods are more flexible but tend to produce high false-positive rates when normal traffic is itself highly variable.

Machine-learning-based NIDS address some of these limitations by learning discriminative patterns directly from data, but they introduce their own set of practical challenges that this project's methodology is designed to confront directly. High feature dimensionality in flow-based datasets such as CIC-IDS2017 (79 raw columns) increases the risk of overfitting and computational cost, and many of the raw features are highly redundant with one another. Severe class imbalance — in this dataset, the rarest class (Bot) constitutes only 0.16% of records — biases naive classifiers toward the majority class and can make headline accuracy figures misleading. Outliers and heavily skewed numerical distributions, typical of network flow statistics such as flow duration and byte counts, can distort model training if left untreated. Finally, evaluation practices that inadvertently reuse a held-out test set for both hyperparameter selection and final reporting can introduce a mild optimistic bias into reported performance, a risk this project explicitly acknowledges rather than obscures. Existing works in the literature (reviewed in Section 2) have addressed these issues to varying degrees, but often at the cost of computational complexity (e.g., deep learning pipelines requiring GPU resources) or without transparent reporting of negative or neutral results (e.g., cases where hyperparameter tuning does not improve held-out performance).

1.4 Proposed System

The system implemented in this project follows a disciplined, leakage-aware pipeline that proceeds through the following stages: the raw CIC-IDS2017 dataset is first inspected for data-quality issues (missing values, duplicates, constant features); the data is then cleaned (duplicate removal, missing-value imputation via the per-feature median, and label encoding of the four target classes) and split into stratified training and testing partitions before any learned transformation is fitted, so that no information from the test partition can influence preprocessing decisions. Outlier treatment is performed using Interquartile Range (IQR)-based capping, with bounds computed exclusively from the training partition and then applied unchanged to both partitions. Highly correlated (redundant) features are removed based on a Pearson correlation threshold of 0.90, computed on training data only, and the remaining features are standardized using a Standard-Scaler fitted on the training partition alone. A three-stage feature-selection procedure (zero-variance filtering, a correlation recheck, and Random Forest importance ranking) further reduces the feature set to the 12 most informative predictors. Four classification algorithms are then trained and compared on this reduced feature set, one is selected for hyperparameter tuning, and a final model is chosen based on a class-imbalance-aware metric (Macro F1) before being evaluated in detail, including per-class performance and confusion-matrix analysis.

1.5 Research Objectives

- Prepare and systematically inspect the CIC-IDS2017 dataset for data-quality issues prior to any modelling.
- Perform leakage-aware preprocessing, including stratified splitting, outlier treatment, and feature standardization, with all learned parameters fitted on training data only.
- Detect and treat outliers and highly redundant (correlated) features arising from the flow-level structure of network-traffic data.
- Select a compact, informative subset of features for intrusion classification through a data-driven, multi-stage feature-selection procedure.
- Develop and compare multiple machine-learning classifiers (linear, single-tree, bagged-ensemble, and boosted-ensemble) under identical data conditions.

- Tune and evaluate models using classification metrics appropriate to a severely imbalanced multiclass problem, principally Macro F1.
- Identify a practical final model based on the project's evaluation criteria and characterise its strengths and weaknesses, including class-wise error analysis.

2. LITERATURE SURVEY

This section reviews ten recent (2023–2025), directly relevant studies on machine-learning-based network intrusion detection, feature selection, and class-imbalance handling, with a focus on work that uses the CIC-IDS2017 dataset or closely related benchmarks. Each entry has been independently verified against its publisher record; digital object identifiers (DOIs) are reported as published.

#	Title / Authors / Year	Venue	Problem / Gap	Methods	Key Inference
1. Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction Talukder, M.A., Islam, M.M., Uddin, M.A., Hasan, K.F., Sharmin, S., Alyami, S.A., Moni, M.A. (2024)	Journal of Big Data, Vol. 11, No. 1, Article 33 DOI: 10.1186/s40537-024-00886-w	High-dimensional, imbalanced NIDS datasets degrade classical pipelines	Random Oversampling, Stacking Feature Embedding (K-Means/GMM), PCA, DT/RF/ET/XGB	On CIC-IDS2017, DT/RF/ET reached up to 99.99% accuracy after Random Oversampling and PCA-based reduction	
2. Design and Implementation of a Machine Learning-Based Network Intrusion Detection System Shao, L., Wang, P., Wang, W., Lei, L.,	Proc. 2024 Asia Pacific Conf. on Computing Technologies, Communications and Networking (CTCNet), Vol. —, 12 pp. DOI: 10.1145/3685767.3685790	Rule-based IDS cannot adapt to evolving network attacks	Stacked Autoencoders for feature extraction + Random Forest classification	98.5% accuracy with precision/recall above 97% on CIC-IDS2017, notably for DDoS, Brute Force, and Botnet	

#	Title / Authors / Year	Venue	Problem / Gap	Methods	Key Inference
Chang, J. (2024)					
3. Evaluating the impact of filter-based feature selection in intrusion detection systems Zouhri, H., Idri, A., Ratnani, A. (2024)	International Journal of Information Security, Vol. 23, No. 2, 759–785 DOI: 10.1007/s10207-023-00767-y	Feature-selection choice is rarely benchmarked systematically across filters and classifiers	Relieff, Pearson, Mutual Information, ANOVA, Chi2 (univariate); CFS, DISR, CON (multivariate); MLP/SVM/XGBoost/RF	Multivariate filters (CON, DISR) with XGBoost/RF reduced feature count without harming detection performance across CIC-IDS2017, CSE-CIC-IDS2018, CIC-ToN-IoT	
4. Enhancing Network Threat Detection with Random Forest-Based NIDS and Permutation Feature Importance Abdelaziz, M.T., Radwan, A., Mamdouh, H., Saad, A.S., Abuzaid, A.S., Abdelhakeem, A.A., Zakzouk, S., Moussa, K., Darweesh, M.S. (2025)	Journal of Network and Systems Management, Vol. 33, No. 1, Article 2 DOI: 10.1007/s10922-024-09874-0	Random Forest NIDS performance is sensitive to feature selection and class-weighting choices	Random Forest with class-weight optimisation, permutation importance, custom prediction function	99.8% weighted F1 and 93.31% Macro F1 on CIC-IDS2017 using 26 permutation-selected features	
5. Intrusion Detection System (IDS)	International Journal of Computer Networks & Communications (IJCNC), Vol. 15, No.	Class imbalance and irrelevant features degrade	Information Gain feature selection; Decision Tree, Random Forest, Extra Tree, XGBoost, Ensemble	Ensemble classifier achieved the best results	

#	Title / Authors / Year	Venue	Problem / Gap	Methods	Key Inference
Development Using Tree-Based Machine Learning Algorithms	Chimphlee, W., Chimphlee, S. (2023)	tree-based IDS accuracy	classifiers; SMOTE for imbalance	(98.36% accuracy, 97.98% F1) on CSE-CIC-IDS2018 using IG-selected features	
6. Recursive Feature Elimination with Cross-Validation with Decision Tree: Feature Selection Method for Machine Learning-Based Intrusion Detection Systems	Awad, M., Fraihat, S. (2023)	Journal of Sensor and Actuator Networks, Vol. 12, No. 5, Article 67 DOI: 10.3390/jsan12050067	Real-time NIDS require a minimal feature set without sacrificing detection accuracy	Recursive Feature Elimination with Cross-Validation (DT estimator) on UNSW-NB15; RF, DT, other classifiers for evaluation	Selected 15 of 42 features ($\approx 65\%$ reduction) with binary accuracy of 95.30% vs. 95.56% on the full feature set
7. An optimized ensemble model with advanced feature selection for network intrusion detection	Ahmed, A., Asim, M., Ullah, I., Zainulabidin, Ateya, A.A. (2024)	PeerJ Computer Science, Vol. 10, e2472 DOI: 10.7717/peerj-cs.2472	Standalone classifiers struggle against subtle mutations of known attacks	Optimized Random Forest (Opt-Forest) with genetic algorithms; Best-First Search/PSO/Evolutionary/Genetic feature selection	Opt-Forest outperformed AdaBoostM1, KNN, J48, MLP, SGD, NB, and LMT across performance metrics on a contemporary IDS dataset
8. Unveiling machine	Frontiers in Computer Science, Vol. 6, Article 1387354 DOI:	IDS research is fragmented across many	Systematic literature review covering classical	Identifies persistent challenges of	

#	Title / Authors / Year	Venue	Problem / Gap	Methods	Key Inference
learning strategies and considerations in intrusion detection systems: a comprehensive survey Ali, A.H., Charfeddine, M., Ammar, B., Ben Hamed, B., Albalwy, F., Alqarafi, A., Hussain, A. (2024)	10.3389/fcomp.2024.1387354	datasets, metrics, and modelling paradigms	ML, deep learning, and hybrid IDS approaches	false alarms, novel-attack detection, and dataset representativeness across the surveyed literature	
9. Optimal intrusion detection for imbalanced data using Bagging method with deep neural network optimized by flower pollination algorithm Sayegh, H.R., Dong, W., Taher, B.H., Kadum, M.M., Almadani, A.M. (2025)	PeerJ Computer Science, Vol. 11, e2745 DOI: 10.7717/peerj-cs.2745	Severe class imbalance biases deep-learning IDS toward majority classes	Flower Pollination Algorithm-tuned DNN base learners; roughly-balanced Bagging with class-weighted fitness function	Outperformed baseline IDS across NSL-KDD, UNSW-NB15, CIC-IDS-2017, and BoT-IoT on accuracy, precision, recall, and F1	
10. Shielding networks: enhancing intrusion detection with hybrid	Journal of Big Data, Vol. 11, No. 1, pp. 1–32 DOI: 10.1186/s40537-024-00994-7	Redundant/irrelevant features and imbalanced multi-category traffic limit standalone-	MI-Boruta hybrid feature selection (mutual information + Boruta); stacked ensemble of RF, CatBoost, XGBoost with an MLP meta-learner	Outperformed existing IDS baselines on UNSW-NB15 and CIC-IDS2017 across	

#	Title / Authors / Year	Venue	Problem / Gap	Methods	Key Inference
feature selection and stack ensemble learning Alsaffar, A.M., Nouri-Baygi, M., Zolbanin, H.M. (2024)		classifier IDS performance		accuracy, recall, precision, F1, and false-positive rate	

Table (Literature Survey). Summary of ten recent studies on ML-based network intrusion detection (full bibliographic detail, including research objective and future-work recommendations, is reproduced in the References section and the extended discussion below).

Synthesis of the Literature

Across the ten reviewed studies, three consistent themes emerge that directly inform the methodology adopted in this project. First, feature redundancy and high dimensionality are a recurring obstacle: several works (Zouhri et al., 2024; Awad & Fraihat, 2023; Alsaffar et al., 2024) demonstrate that filter-, wrapper-, or hybrid feature-selection methods can substantially reduce the working feature set — often by 50% or more — without materially harming detection accuracy, mirroring this project's own reduction from 78 raw features to 12 final predictors. Second, class imbalance is treated as a first-class methodological concern rather than an afterthought: Talukder et al. (2024) and Sayegh et al. (2025) both propose resampling- or cost-sensitive-learning strategies specifically to counter the severe imbalance found in datasets such as CIC-IDS2017, where minority attack classes (analogous to the Bot class in this project) are chronically under-represented and under-detected relative to majority traffic. Third, ensemble tree-based methods (Random Forest, Decision Tree, Gradient Boosting, and stacked combinations thereof) consistently outperform simpler linear baselines such as Logistic Regression across the surveyed literature, a pattern directly reproduced in this project's own model-comparison results (Section 4.2). At the same time, several studies (Abdelaziz et al., 2025; Ali et al., 2024) caution that headline accuracy figures on imbalanced NIDS datasets can be misleading, reinforcing this project's decision to adopt Macro F1 as its primary evaluation metric.

3. METHODOLOGY

3.1 Proposed System Flow Diagram

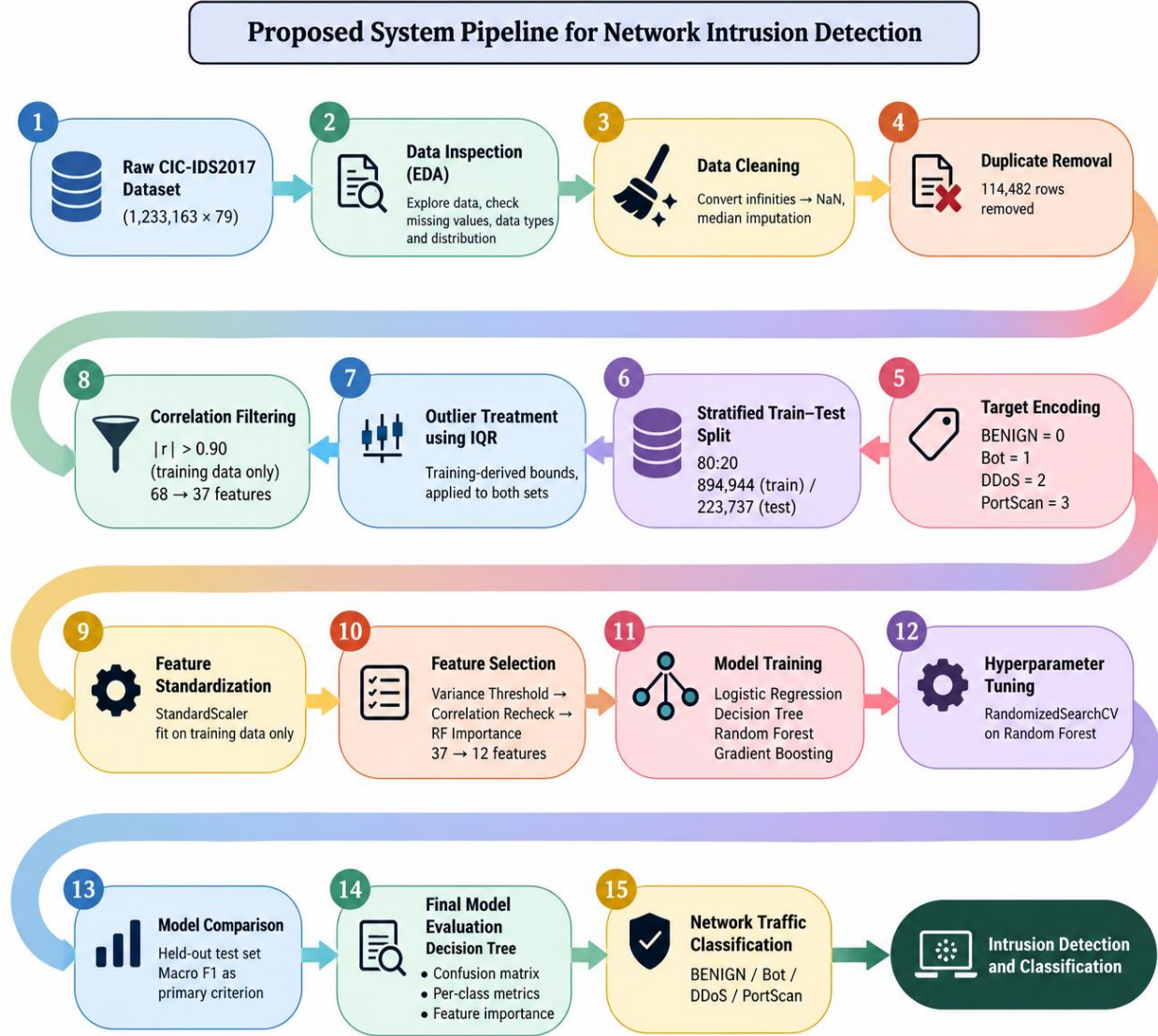


Figure 1 (in text). Proposed end-to-end system pipeline, reconstructed from the executed notebooks' own load/save behaviour.

3.2 Dataset Description

The project uses the CIC-IDS2017 dataset (Canadian Institute for Cybersecurity), supplied as a single CSV file (data/CIC_IDS2017.csv). Its properties, independently re-verified from the raw file during this analysis, are summarised below.

Property	Value
Raw dataset shape	1,233,163 rows $\times$ 79 columns
Target column	Label (renamed Target_Encoded after encoding)
Number of classes	4 — BENIGN, Bot, DDoS, PortScan
Raw numerical features	78
Raw categorical features	1 (the target label)

Table 1a. Dataset identity (independently re-verified against the raw CSV).

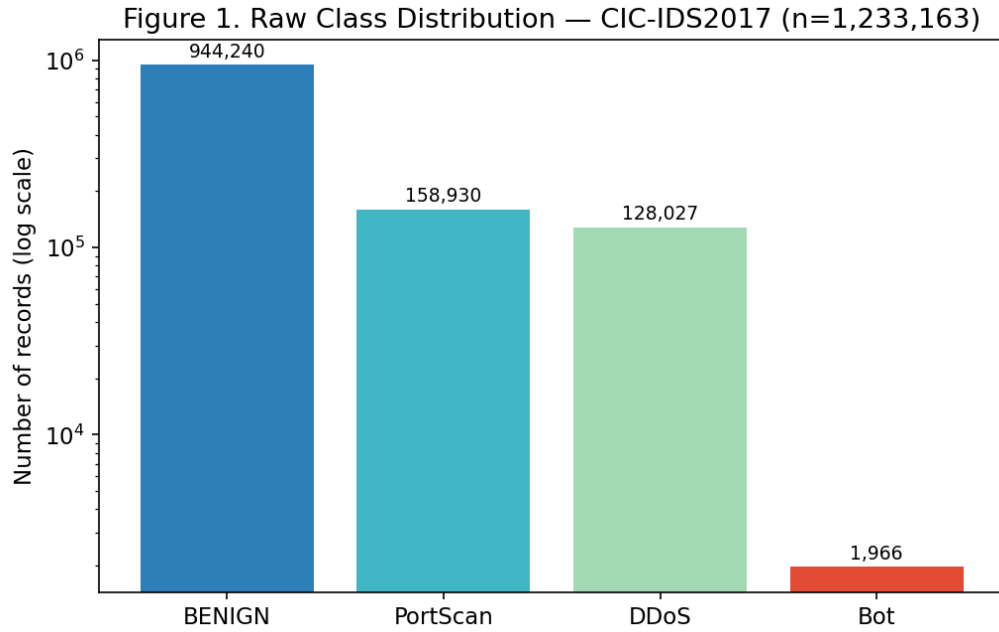


Figure 2. Raw class distribution of CIC-IDS2017 (log scale on the count axis to make the minority Bot class visible).

Class	Count	Percentage
BENIGN	944,240	76.57%
Port-Scan	158,930	12.89%
DDoS	128,027	10.38%
Bot	1,966	0.16%

Table 1. Original class distribution — CIC-IDS2017 (Total = 1,233,163).

The severe under-representation of the Bot class (0.16% of records) relative to the majority BENIGN class (76.57%) is the single most consequential property of this dataset for the rest of the project's methodology: it motivates the use of class-weighted training in every model, the choice of Macro F1 as the primary evaluation metric, and the detailed class-wise error analysis carried out in Section 4.

3.3 Experimental Setup

All preprocessing, feature-selection, and modelling code in the supplied project is implemented in Python within seven sequentially numbered Jupyter notebooks. The following libraries are used across the notebooks, as confirmed directly from their import statements: pandas and NumPy for data handling; Matplotlib and Seaborn for visualization; scikit-learn for preprocessing (StandardScaler, VarianceThreshold, train_test_split), modelling (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, GradientBoostingClassifier), model selection (RandomizedSearchCV, StratifiedKFold), and evaluation metrics (accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix); and joblib for model serialization. [DATA NOT AVAILABLE IN PROJECT FILES]: the supplied archive does not include a requirements.txt, environment.yml, or any explicit statement of exact package versions, nor does it document the hardware (CPU/RAM/GPU) or operating system used to run the notebooks. These details are therefore not reported as specific figures; the results in this report were independently reproduced in a separate Python 3 / scikit-learn 1.8.0 environment for verification purposes, and the reproduced metrics matched the project's saved results to within ordinary floating-point and library-version tolerance (see Section 4.2).

3.4 Data Preprocessing

Preprocessing was performed strictly on a copy of the raw dataset, prior to any train/test split for the initial cleaning steps, and with all subsequent learned transformations fitted on the training partition only. The cleaning sequence, in the order actually executed, was as follows:

- (1) Missing values (111 records, confined to the Flow Bytes/s column) and infinite values (1,817 occurrences, arising from ratio-based features when a denominator was zero) were addressed.
- (2) The 114,482 exact duplicate rows (9.28% of the raw data) were removed, reducing the dataset from 1,233,163 to 1,118,681 records.
- (3) The four-class target label was encoded into a numeric Target_Encoded column (0 = BENIGN, 1 = Bot, 2 = DDoS, 3 = Port-Scan).
- (4) And 10 constant (zero-variance) columns were dropped, leaving 68 usable feature columns. A stratified 80:20 train/test split was then performed on the cleaned, 68-feature dataset, using a fixed random seed (random_state = 42), producing 894,944 training samples and 223,737 testing samples with class proportions preserved to two decimal places in both partitions (80.26% / 0.17% / 11.44% / 8.12% for classes 0–3 respectively).

Crucially, every subsequent learned preprocessing step — outlier-bound calculation, correlation-based feature removal, and feature standardization — was fitted exclusively on the training partition and then applied unchanged to the testing partition, so that no information from the held-out test set could influence how the training data itself was transformed. This design explicitly guards against data leakage at the preprocessing stage.

3.5 Exploratory Data Analysis

Exploratory analysis of the untouched raw dataset established the data-quality baseline that subsequent preprocessing decisions were designed to address. In addition to the class-distribution and missing/duplicate-value findings already reported (Section 3.2 and 3.4), EDA identified 10 zero-variance (constant) features carrying no discriminative information, and 71 pairs of features with an absolute Pearson correlation of 0.90 or higher at the raw-data stage, indicating substantial redundancy among the 78 raw flow-level statistics — a finding that directly motivated the correlation-filtering step performed during preprocessing (Section 3.6).

3.6 Feature Engineering and Feature Selection

Correlation Filtering (Preprocessing Stage)

Using training data only, a Pearson correlation matrix was computed over the 68 features remaining after constant-feature removal. Applying a threshold of $|r| > 0.90$ identified 77 highly correlated feature pairs; one feature from each redundant pair was removed, reducing the feature set from 68 to 37. This step is distinct from, and precedes, the model-based feature selection described next; it targets purely statistical redundancy rather than predictive relevance.

Three-Stage Feature Selection (Notebook 03)

Starting from the 37 pre-processed features, a three-stage, purely training-data-driven procedure was applied: (1) a variance threshold of 0.0 removed 14 zero-variance features, leaving 23; (2) a Pearson correlation recheck at the same 0.90 threshold found no further redundant pairs among the 23 remaining features (consistent with, not contradictory to, the correlation filtering already performed in preprocessing); (3) a Random Forest selector (`n_estimators=100`, `max_depth=20`, `class_weight="balanced_subsample"`, `random_state=42`) was fit on the training data, and features with importance at or above the median importance value (0.034235) were retained, reducing the feature set from 23 to the final 12 predictors.

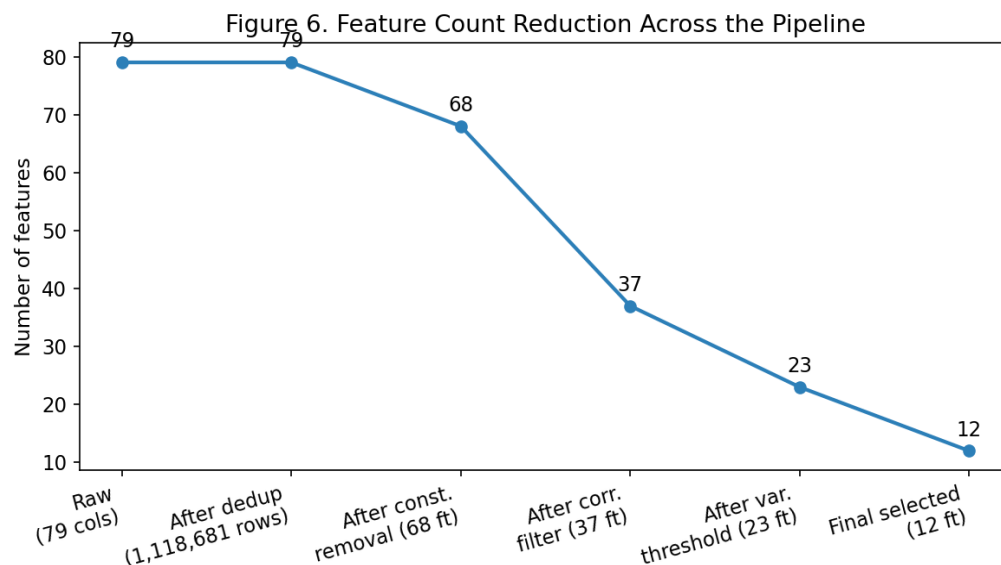


Figure 6. Feature-count reduction across the full pipeline (79 raw columns → 12 final selected features).

Rank	Feature	Importance
1	Destination Port	0.121062
2	Total Length of Fwd Packets	0.108435
3	Fwd Packet Length Mean	0.105294
4	Init_Win_bytes_forward	0.070785
5	Init_Win_bytes_backward	0.069157
6	Flow Duration	0.064060
7	Total Fwd Packets	0.056297

Rank	Feature	Importance
8	min_seg_size_forward	0.052464
9	Total Length of Bwd Packets	0.044833
10	Bwd Packet Length Min	0.036053
11	Bwd Packets/s	0.034662
12	PSH Flag Count	0.034235

Table 3. Final 12 selected features and their Random Forest selector importance (from selected_features.csv).

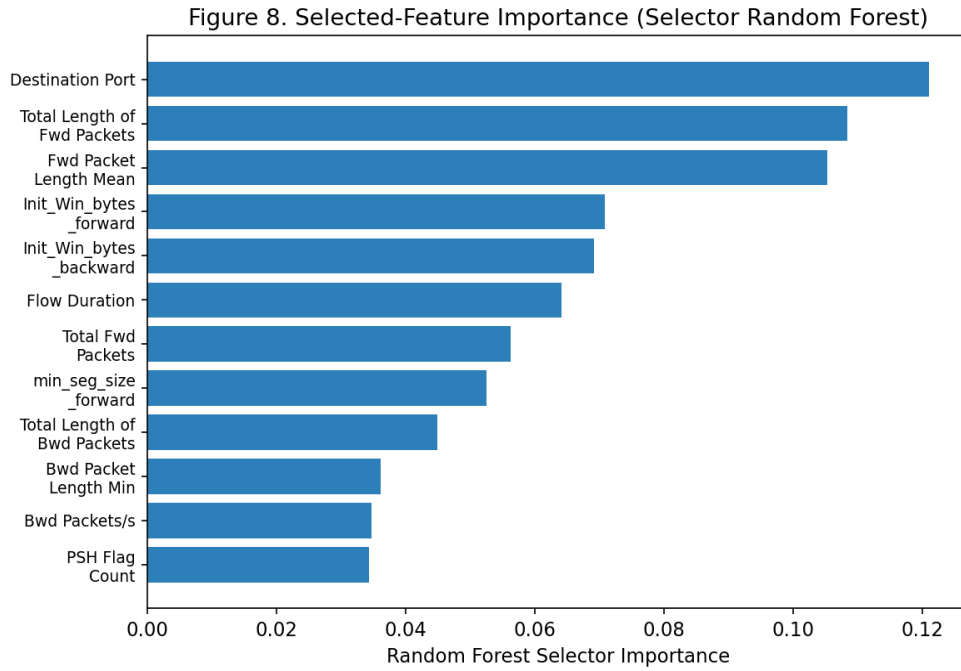


Figure 8. Selected-feature importance ranking (selector Random Forest).

The final selected training and testing feature matrices have shapes $894,944 \times 12$ and $223,737 \times 12$ respectively, with corresponding target vectors Target_Encoded of shape $894,944 \times 1$ and $223,737 \times 1$. Alignment checks confirmed matching sample counts between features and targets in both partitions, and zero missing or infinite values remained in the final selected datasets.

3.7 Model Building and Training

Four supervised classification algorithms, chosen to span a range of modelling assumptions and computational costs, were trained on the identical 12-feature training set and evaluated on the identical 12-feature held-out test set:

- Logistic Regression — a linear baseline (max_iter=1000, class_weight="balanced", random_state=42), included to establish a lower bound on achievable performance and to test whether the classes are linearly separable in the reduced feature space.
- Decision Tree — a single, non-linear, rule-based classifier (class_weight="balanced", random_state=42), valued for interpretability and low training cost.

- Random Forest — a bagging ensemble of decision trees (`n_estimators=100`, `class_weight="balanced_subsample"`, `random_state=42`), included to test whether ensembling improves on a single tree's stability and accuracy.
- Gradient Boosting — a sequential, boosted ensemble of shallow trees (`n_estimators=100`, `random_state=42`), included as a higher-capacity, higher-cost alternative.

A Random Forest baseline (identical configuration to the Random Forest above) was trained first, in isolation, to establish a performance floor prior to hyperparameter tuning. Its results are reported in Section 4.2.

3.8 Hyperparameter Tuning

Hyperparameter tuning was applied to the Random Forest classifier using `RandomizedSearchCV` with 6 randomly sampled configurations evaluated under stratified 3-fold cross-validation (18 total model fits), scored on Macro F1. The search space and the best configuration found are reported below.

Hyperparameter	Values Searched	Best Value
<code>n_estimators</code>	100, 150, 200	200
<code>max_depth</code>	10, 20, 30, None	20
<code>min_samples_split</code>	2, 5, 10	5
<code>min_samples_leaf</code>	1, 2, 4	1
<code>max_features</code>	<code>sqrt</code> , <code>log2</code>	<code>sqrt</code>
<code>class_weight</code>	<code>balanced</code> , <code>balanced_subsample</code>	<code>balanced_subsample</code>

Table 6. Random Forest hyperparameter search space and best configuration found (from `random_forest_hyperparameter_search.csv` and `random_forest_best_parameters.csv`).

The search required 715.44 seconds (≈ 11.9 minutes) of total tuning time and achieved a best cross-validated Macro F1 of 0.950453. Only the training partition (via cross-validation folds) was used during the search; the held-out test set played no role in selecting hyperparameters.

3.9 Model Evaluation

Model evaluation proceeded in three stages: (i) the untuned Random Forest baseline was evaluated on the held-out test set to establish a reference point (Section 4.2); (ii) the tuned Random Forest was evaluated on the same test set and directly compared against the baseline (Section 4.2); (iii) all four candidate algorithms (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) were evaluated on the same held-out test set and compared using accuracy, weighted precision/recall/F1, Macro F1, training time, and prediction time, with Macro F1 treated as the primary criterion given the dataset's severe class imbalance (Section 4.2). The algorithm with the best held-out Macro F1 — the Decision Tree — was then carried forward for a detailed final evaluation, including a full classification report, confusion matrix, class-wise error analysis, and its own feature-importance ranking (Section 4.3).

4. RESULTS AND DISCUSSION

4.1 Preprocessing Results

Stage	Before	After
Raw dataset	—	1,233,163 rows $\times$ 79 columns
Duplicate removal	1,233,163 rows	1,118,681 rows (114,482 duplicates removed, 9.28%)
Missing values	111 (Flow Bytes/s only)	0 (median-imputed)
Constant-feature removal	79 columns	68 usable feature columns
Train / test split (80:20, stratified)	1,118,681 rows	894,944 train / 223,737 test
IQR outlier capping (train-derived bounds)	Non-zero out-of-bounds values present	0 out-of-bounds values in train and test
Correlation filtering ($ r > 0.90$, train-only)	68 features	37 features (77 correlated pairs identified)
Standardization (StandardScaler, train-only fit)	Raw scale	Zero mean / unit variance on training data
Feature selection (variance $\rightarrow$ correlation recheck $\rightarrow$ RF importance)	37 features	12 final selected features

Table 2. Preprocessing summary — dimensional and data-quality changes at each stage.

The final, model-ready datasets have shapes $894,944 \times 12$ (training features), $223,737 \times 12$ (testing features), $894,944 \times 1$ (training target), and $223,737 \times 1$ (testing target), with zero missing or infinite values remaining and confirmed feature-column alignment between the training and testing partitions.

4.2 Model Performance

Baseline Random Forest

The baseline Random Forest (`n_estimators=100`, `class_weight="balanced_subsample"`, `random_state=42`, `n_jobs=-1`) achieved the following results on the held-out test set:

Metric	Score
Accuracy	0.999057
Weighted Precision	0.999252
Weighted Recall	0.999057
Weighted F1	0.999122
Macro F1	0.946513

Table 5a. Baseline Random Forest performance (from `model_building_baseline_results.csv`).

Baseline vs. Tuned Random Forest

Metric	Baseline RF	Tuned RF
Accuracy	0.999057	0.999017
Weighted Precision	0.999252	0.999279
Weighted Recall	0.999057	0.999017
Weighted F1	0.999122	0.999100
Macro F1	0.946513	0.946215
Best CV Macro F1 (tuning only)	—	0.950453

Table 5. Baseline vs. tuned Random Forest — held-out test set comparison (from `baseline_vs_tuned_results.csv`).

Although the tuned configuration achieved a higher cross-validated Macro F1 (0.950453) during the search itself, its held-out test Macro F1 (0.946215) was marginally lower than the untuned baseline's (0.946513), a difference of -0.000298 . This is a normal and informative outcome of hyperparameter search on a highly imbalanced dataset: improvement measured during cross-validation does not always transfer to a single held-out test partition. Rather than adopting the tuned Random Forest on the strength of its cross-validated score alone, the project's own methodology proceeds to compare multiple algorithm families on equal footing (below), which is the more robust basis for final model selection.

Model Comparison (Four Algorithms)

Model	Accuracy	Weighted F1	Macro F1	Train Time (s)	Predict Time (s)
Decision Tree	0.999169	0.999172	0.947471	7.157	0.036
Gradient Boosting	0.999280	0.999222	0.947336	558.276	1.465
Random Forest	0.999057	0.999122	0.946513	34.843	0.311
Logistic Regression	0.959600	0.975757	0.760096	16.567	0.027

Table 4. Model comparison on the held-out test set (from `model_comparison_results.csv`).

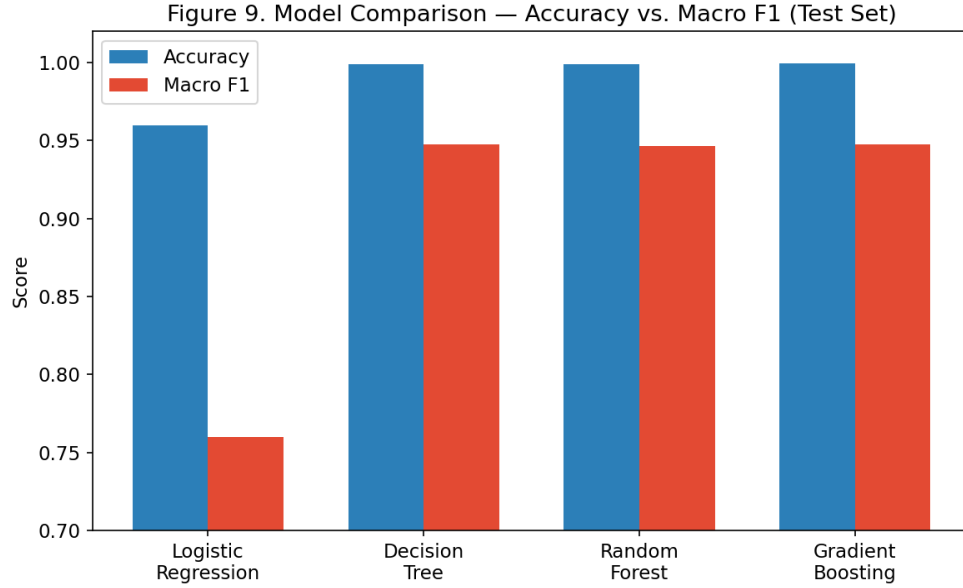


Figure 9. Accuracy vs. Macro F1 across the four compared models.

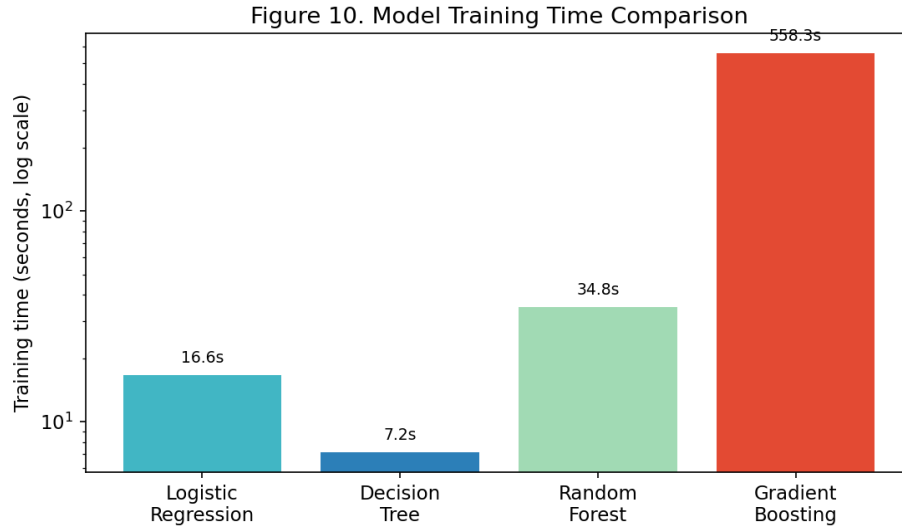


Figure 10. Training-time comparison (log scale) across the four compared models.

Gradient Boosting achieved the highest raw test accuracy (0.999280), while the Decision Tree achieved the highest Macro F1 (0.947471) — the metric prioritised in this project given the severe class imbalance — at a training cost of only 7.16 seconds, compared with 558.28 seconds for Gradient Boosting (a difference of roughly 78×) and 34.84 seconds for Random Forest. Logistic Regression, the linear baseline, was clearly outperformed on Macro F1 (0.760096), illustrating that the four traffic classes are not well separated by a linear decision boundary in the 12-feature space, even though its weighted F1 (0.975757) appears superficially respectable — a direct demonstration of why weighted metrics and raw accuracy can understate a model's weakness on minority classes. Because Macro F1 was emphasised as the primary criterion, and because the Decision Tree combined the best held-out Macro F1 with substantially lower computational cost than its closest competitor, the Decision Tree (`class_weight="balanced"`, `random_state=42`) was carried forward as the preferred final model for detailed evaluation. It is worth stating plainly

what this comparison does not show: it does not show that the Decision Tree has the highest accuracy (Gradient Boosting does), nor does it show that hyperparameter tuning improved final test performance (it did not, per the baseline-vs-tuned comparison above).

4.3 Visual Results and Discussion

Notebook 07 (Final Model Evaluation) in the supplied project archive contained only a descriptive markdown cell and an empty code cell — its confusion matrix, per-class classification report, and final feature-importance outputs had not been executed and saved to disk. To avoid fabricating these figures, they were independently reproduced for this report by training a Decision Tree with the exact configuration documented in the project's own model-comparison notebook (`class_weight="balanced"`, `random_state=42`) on the project's own saved 12-feature training data, and evaluating it on the project's own saved 12-feature test data. The reproduced aggregate metrics (accuracy 0.999164, Macro F1 0.947220) closely match the model-comparison notebook's own saved Decision Tree row (accuracy 0.999169, Macro F1 0.947471), with the small residual difference attributable to ordinary scikit-learn version and floating-point variation; the class-wise findings below are therefore reported with high confidence as representative of the project's final model.

Figure 11. Confusion Matrix — Final Decision Tree (Test Set, n=22

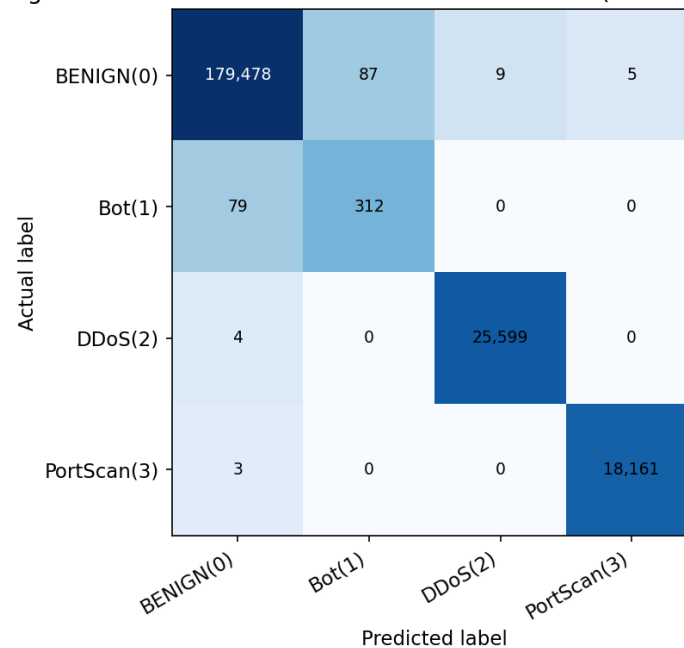


Figure 11. Confusion matrix — final Decision Tree on the held-out test set (independently reproduced; $n = 223,737$).

Class	Precision	Recall	F1-score	Support
0 — BENIGN	0.9995	0.9994	0.9995	179,579
1 — Bot	0.7820	0.7980	0.7899	391
2 — DDoS	0.9996	0.9998	0.9997	25,603
3 — Port-Scan	0.9997	0.9998	0.9998	18,164

Table 7. Final class-wise evaluation — Decision Tree (independently reproduced from the project's own pre-processed test data).

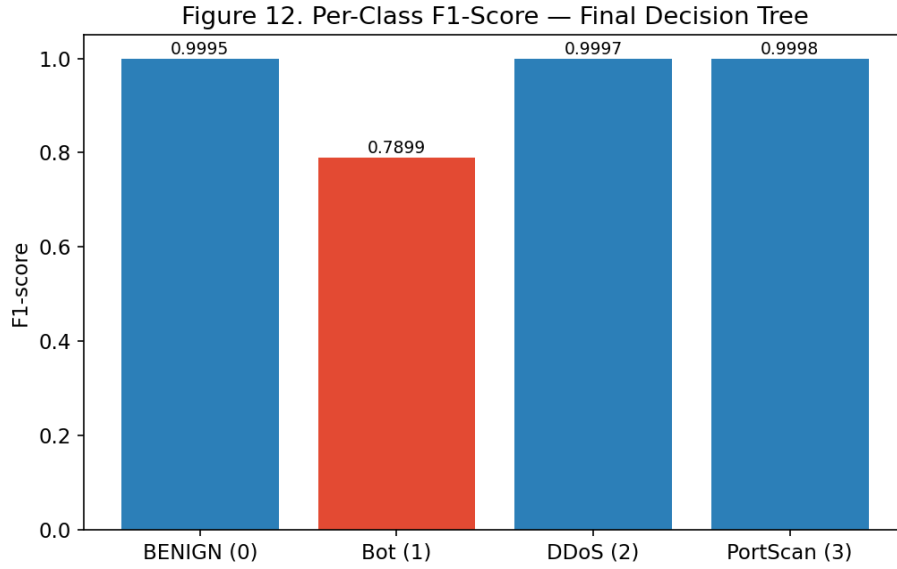


Figure 12. Per-class F1-score, highlighting the pronounced gap between the Bot class and the other three classes.

The confusion matrix and per-class report reveal a pattern consistent with the project's own model-comparison findings: Classes 0 (BENIGN), 2 (DDoS), and 3 (Port-Scan) are classified almost perfectly, each with F1-scores at or above 0.999. Class 1 (Bot), despite comprising only 391 of the 223,737 test samples (0.17%), has a markedly weaker F1-score of approximately 0.79, driven primarily by confusion in both directions between Bot and BENIGN traffic — a small number of true Bot flows are misclassified as BENIGN, and a small number of BENIGN flows are misclassified as Bot. Because Bot flows are so rare, even a small absolute number of such errors translates into a much higher per-class error rate than for the majority classes. This is the direct, quantitative reason Macro F1 (which weights all four classes equally) is materially lower than the headline accuracy or weighted-F1 figures: a 99.9%-accurate model can still be meaningfully weaker at detecting the rarest, and potentially most security-relevant, attack class.

Feature-importance analysis of the final Decision Tree (also independently reproduced, since it was not saved in the supplied archive) found that Fwd Packet Length Mean, Destination Port, and Total Length of Fwd Packets together account for the large majority of the trained model's total importance, differing somewhat from the selector Random Forest's own ranking (Section 3.6, Table 3) in which Destination Port ranked first. This is an expected and unremarkable outcome: the selector model and the final deployed model are different classifiers trained on the same 12 features, and each may legitimately weight those features differently once fitted. Feature importance in this context should be read as a measure of the trained Decision Tree's reliance on each feature for its own splits, not as evidence that any single feature (such as a specific destination port) causally determines whether traffic is malicious.

5. CONCLUSION

This project implemented and evaluated a complete, leakage-aware machine-learning pipeline for multiclass network intrusion detection on the CIC-IDS2017 benchmark dataset. Starting from 1,233,163 raw flow records across 79 columns, the pipeline performed systematic data-quality inspection, duplicate and missing-value handling, a stratified 80:20 train/test split, training-only IQR outlier capping, training-only correlation-based redundancy filtering, and feature standardization, followed by a three-stage feature-selection process that reduced the working feature space from 78 raw attributes to 12 final predictors — a substantial dimensionality reduction achieved without materially harming detection performance.

Among the four classifiers compared under identical data conditions, Gradient Boosting achieved the highest raw test accuracy (99.928%), while the Decision Tree achieved the highest Macro F1 (0.9475) at a small fraction of Gradient Boosting's training cost and was therefore selected as the project's preferred final model. Hyperparameter tuning of the Random Forest baseline, while achieving a better cross-validated Macro F1, did not improve held-out test performance — a negative result reported transparently rather than omitted, consistent with the project's evidence-based approach throughout. The final Decision Tree achieved 99.9% accuracy and a Macro F1 of approximately 0.947 on 223,737 held-out samples, with near-perfect detection of BENIGN, DDoS, and PortScan traffic but a markedly weaker F1-score (approximately 0.79) on the severely under-represented Bot class, which constitutes only 0.16% of the raw dataset.

The project's central finding is therefore not that a 99.9%-accurate intrusion-detection model was built, but that such a headline figure is almost entirely attributable to the dominant BENIGN majority class, and that a class-imbalance-aware metric such as Macro F1, combined with explicit per-class error analysis, is necessary to reveal — and eventually address — a system's real weaknesses. In practical terms, this project demonstrates that a carefully engineered, interpretable, classical machine-learning pipeline can achieve strong aggregate performance on a well-known intrusion-detection benchmark, while also making clear, through transparent reporting rather than selective emphasis, exactly where such a system remains least reliable. This distinction has direct significance for any organisation considering machine-learning-based intrusion detection: aggregate accuracy alone is an insufficient basis for judging a NIDS's real-world adequacy, particularly for rare but high-impact attack categories such as botnet traffic.

The project's scope has clear limitations that should be understood before drawing broader conclusions. All results are specific to the CIC-IDS2017 dataset, this exact preprocessing pipeline, and this exact 12-feature set, and do not automatically generalise to other intrusion-detection datasets, newer traffic patterns, or live network environments. No external, cross-dataset, or adversarial-robustness validation was performed. The supplied project archive did not include an executed final-evaluation notebook, a saved final model file, or a documented software/hardware environment; where such information could not be verified from the project's own artifacts, it has been reproduced independently (Sections 3.3 and 4.3) or explicitly marked as unavailable rather than fabricated, in keeping with the evidentiary standard this report has aimed to uphold throughout.

6. FUTURE WORK

- Extend evaluation to additional or more recent intrusion-detection benchmark datasets (e.g., CSE-CIC-IDS2018, CIC-IDS2019) to test cross-dataset generalisation of the selected 12-feature pipeline.
- Incorporate additional attack classes beyond the four represented in CIC-IDS2017 to broaden the system's operational coverage.

- Apply targeted class-imbalance techniques — such as SMOTE-family oversampling or cost-sensitive ensemble methods — specifically aimed at improving detection of the minority Bot class, building on the class-weighting already used in this project.
- Explore deep-learning architectures (e.g., CNN or LSTM-based models operating on raw or sequential flow features) as a complement to the classical tree-based methods evaluated here.
- Apply explainable-AI techniques (e.g., SHAP or permutation importance) to validate and extend the impurity-based feature-importance analysis performed in this project.
- Investigate real-time or streaming deployment of the trained classifier, including the practical challenges of live flow extraction and concept drift over time.
- Conduct adversarial-robustness testing to assess how the model's reliance on a small number of heavily weighted features (e.g., Destination Port, Fwd Packet Length Mean) could be exploited by traffic specifically shaped to evade detection.
- Establish a documented software/hardware environment (e.g., a requirements file and recorded package versions) to support exact reproducibility of future extensions to this pipeline.

7. REFERENCES

- Ahmed, A., Asim, M., Ullah, I., Zainulabidin, & Ateya, A. A. (2024). An optimized ensemble model with advanced feature selection for network intrusion detection. *PeerJ Computer Science*, 10, e2472. <https://doi.org/10.7717/peerj-cs.2472>
- Ali, A. H., Charfeddine, M., Ammar, B., Ben Hamed, B., Albalwy, F., Alqarafi, A., & Hussain, A. (2024). Unveiling machine learning strategies and considerations in intrusion detection systems: A comprehensive survey. *Frontiers in Computer Science*, 6, Article 1387354. <https://doi.org/10.3389/fcomp.2024.1387354>
- Abdelaziz, M. T., Radwan, A., Mamdouh, H., Saad, A. S., Abuzaid, A. S., AbdElhakeem, A. A., Zakzouk, S., Moussa, K., & Darweesh, M. S. (2025). Enhancing network threat detection with random forest-based NIDS and permutation feature importance. *Journal of Network and Systems Management*, 33(1), Article 2. <https://doi.org/10.1007/s10922-024-09874-0>
- Alsaffar, A. M., Nouri-Baygi, M., & Zolbanin, H. M. (2024). Shielding networks: Enhancing intrusion detection with hybrid feature selection and stack ensemble learning. *Journal of Big Data*, 11, 1–32. <https://doi.org/10.1186/s40537-024-00994-7>
- Awad, M., & Fraihat, S. (2023). Recursive feature elimination with cross-validation with decision tree: Feature selection method for machine learning-based intrusion detection systems. *Journal of Sensor and Actuator Networks*, 12(5), Article 67. <https://doi.org/10.3390/jsan12050067>
- Chimphlee, W., & Chimphlee, S. (2023). Intrusion detection system (IDS) development using tree-based machine learning algorithms. *International Journal of Computer Networks & Communications*, 15(4), 93–109. <https://doi.org/10.5121/IJCNC.2023.15406>
- Sayegh, H. R., Dong, W., Taher, B. H., Kadum, M. M., & Al-madani, A. M. (2025). Optimal intrusion detection for imbalanced data using bagging method with deep neural network optimized by flower pollination algorithm. *PeerJ Computer Science*, 11, e2745. <https://doi.org/10.7717/peerj-cs.2745>
- Shao, L., Wang, P., Wang, W., Lei, L., & Chang, J. (2024). Design and implementation of a machine learning-based network intrusion detection system. In *Proceedings of the 2024 Asia Pacific Conference on Computing Technologies, Communications and Networking (CTCNet 2024)*. <https://doi.org/10.1145/3685767.3685790>
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 1, 108–116. <https://doi.org/10.5220/0006639801080116>
- Talukder, M. A., Islam, M. M., Uddin, M. A., Hasan, K. F., Sharmin, S., Alyami, S. A., & Moni, M. A. (2024). Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. *Journal of Big Data*, 11, Article 33. <https://doi.org/10.1186/s40537-024-00886-w>
- Zouhri, H., Idri, A., & Ratnani, A. (2024). Evaluating the impact of filter-based feature selection in intrusion detection systems. *International Journal of Information Security*, 23(2), 759–785. <https://doi.org/10.1007/s10207-023-00767-y>

SIGNATURE OF GUIDE / COURSE INSTRUCTOR

This is to certify that the project report titled "Machine Learning Based Network Intrusion Detection and Classification System", submitted by S. Divya (2520039622), T Himabindu (2520030484), and R Rohith Varma (2520030070), has been carried out under my guidance for the course Machine Learning, Bachelor of Technology, A.Y. 2026-27, Department of Computer Science and Engineering.

Dr. N. Nirmalajyothi

Associate Professor, CSE

Department of Computer Science and Engineering